Model checking for temporal logic

Samo Golež Andraž Hribernik Erik Kmetič
Gregor Perčič Jakob Pogačnik Souvent Jan Rutar

University of Ljubljana, Faculty of Mathematics and Physics

April 21, 2026

Table of contents

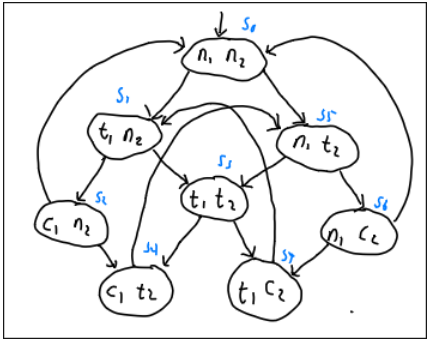
1. Basic definitions
2. LTL model checking
3. Demonstrating a temporal logic model checker
4. Other temporal logics
5. Real-world applications of model checking
6. Turing Award "Model Checking".

Linear-time Temporal Logic

Definition (LTL Transition system)

A *transition system* is given by $(S, \rightarrow, L)$, where

- S is a finite set of *states*
- $\rightarrow \subseteq S \times S$ is the *transition relation*
- $L\colon S \rightarrow \mathcal{P}(\underline{\text{Atoms}})$ is the *labeling function*



Definition (LTL Formulas)

LTL formulas are given by the following grammar:

$$\varphi, \psi ::= p \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \mathbf{X}\varphi \mid \mathbf{F}\varphi \mid \mathbf{G}\varphi \mid \varphi\,\mathbf{U}\,\psi$$

The satisfaction relation

Definition (Run)

An infinite *run* through the transition system $(S, \rightarrow, L)$ is an infinite sequence of states $s_i \in S$

$$s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow \dots$$

such that adjacent states are related by the transition relation.

Definition (The satisfaction relation)

Let π be a run through a transition system and φ a LTL formula, we write the satisfaction relation as

$$\pi \models \varphi$$

The satisfaction relation

$$
\begin{aligned}
\pi \vDash p &\iff p \in L(s_1) \\
\pi \vDash \neg\varphi &\iff \pi \nvDash \varphi \\
\pi \vDash \varphi \wedge \psi &\iff \pi \vDash \varphi \wedge \pi \vDash \psi \\
\pi \vDash \mathbf{X}\varphi &\iff \pi^2 \vDash \varphi \\
\pi \vDash \mathbf{G}\varphi &\iff \forall n \geq 1\ \pi^n \vDash \varphi \\
\pi \vDash \mathbf{F}\varphi &\iff \exists n \geq 1\ \pi^n \vDash \varphi
\end{aligned}
$$

$$
\pi \vDash \varphi\,\mathbf{U}\,\psi \iff (\exists n \geq 1\ \pi^n \vDash \psi) \wedge (\forall m\ 1 \leq m < n\ \pi^m \vDash \varphi)
$$

The model checking problem

Input:

- transition system $\mathcal{M}$,
- starting state s_0,
- LTL formula φ

Output:

- <u>yes</u> if $\pi \models \varphi$ for all runs π through $\mathcal{M}$ starting at s_0
- <u>no</u> otherwise (+ an infinite lasso run)

Language theory

A *word* w over an *alphabet* Σ is a finite sequence of symbols in the set Σ; we write Σ^* for the set of all words over Σ.

Definition (Language)

A *language* L is a set of words $L \subseteq \Sigma^*$

An ω-*word* over an *alphabet* Σ is an infinite sequence of symbols from Σ; we write Σ^ω for the set of all ω-words.

Definition (ω-language)

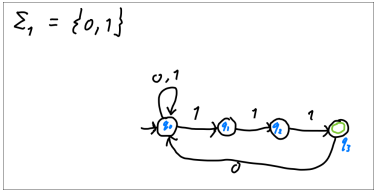
An ω-*language* L is a set of words $L \subseteq \Sigma^\omega$

Büchi automata

Definition (Nondeterministic Büchi automata)

A *nondeterministic Büchi automaton* is a tuple $(\Sigma, Q, I, \Delta, F)$, where

- Σ is an *alphabet*
- Q is the set of *states*
- $I \subseteq Q$ is the set of *initial states*
- $F \subseteq Q$ is the set of *final states*
- $\Delta \colon Q \times \Sigma \to \mathcal{P}(Q)$ is the *transition function*



An automaton A *accepts* an infinite word w if there is some run of A, labeled by w, that goes through the accepting state an infinite number of times. We write $L_\omega(A)$ for the set of all words accepted by A.

Trace

Definition (Trace of a run)

The *trace* of a run $\pi = s_1 \to s_2 \to \ldots$ is the sequence $L(s_1)$, $L(s_2)$, $L(s_3)$, ... this is a word in $(\mathcal{P}(\underline{\text{Atoms}}))^\omega$.

Note that the truth of a formula φ on run π depends only on the trace.

Theorem

The language

$$\mathit{Traces}(\varphi) := \{w \in (\mathcal{P}(\underline{\textit{Atoms}}))^\omega \mid \varphi \textit{ is true on any run with trace } w\}$$

is ω-regular and there exists an algorithm to construct a NBA A_φ recognizing $\mathit{Traces}(\varphi)$.

LTL Model Checking

We want to check $M \models \varphi$ for an LTL formula φ over atomic propositions <u>Atoms</u>.
A run is an infinite sequence $\pi = s_0 s_1 s_2 \cdots$ with $s_i \to s_{i+1}$. Its trace is

$$Trace(\pi) \;=\; L(s_0)\, L(s_1)\, L(s_2) \;\cdots\; \in \;(\mathcal{P}(\underline{\text{Atoms}}))^\omega.$$

NBA operates on *infinite* words $w = a_0\, a_1\, a_2 \;\cdots$. The nondeterministic automaton accepts w iff

$$\exists \text{ run } q_0\, q_1\, q_2 \;\cdots \text{ on } w \text{ such that some } q \in F \text{ appears infinitely often}$$

Given an LTL property φ, we translate $\neg\varphi$ into an NBA $\mathcal{A}_{\neg\varphi}$ whose language is exactly the traces violating φ.

Does M produce any such trace? $\quad M \models \varphi \iff \mathcal{L}(M) \cap \mathcal{L}(\mathcal{A}_{\neg\varphi}) = \emptyset$

$\Rightarrow$ We need to intersect the languages.

Union of NBAs

Given two NBAs $\mathcal{A}_1 = (\Sigma, Q_1, I_1, \Delta_1, F_1)$ and $\mathcal{A}_2 = (\Sigma, Q_2, I_2, \Delta_2, F_2)$:

Union automaton $\mathcal{A}_1 \cup \mathcal{A}_2$:

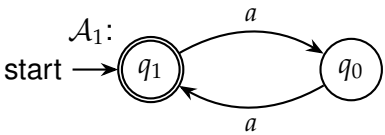
- States: $Q_1 \cup Q_2$ (Q_1, Q_2 disjoint)
- Initial states: $I_1 \cup I_2$
- Transitions: keep Δ_1 and Δ_2
- Accepting states: $F_1 \cup F_2$

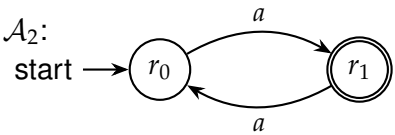
Nondeterministically choose which automaton to run $\Rightarrow$
$\mathcal{L}(\mathcal{A}_1 \cup \mathcal{A}_2) = \mathcal{L}(\mathcal{A}_1) \cup \mathcal{L}(\mathcal{A}_2)$.

Intersection of NBAs — Naive Approach

Same as NFA product:

- States: $Q_1 \times Q_2$
- Initial states: $I_1 \times I_2$
- Transitions: $\Delta((q,r),a) = \Delta_1(q,a) \times \Delta_2(r,a)$
- Accepting: $F_1 \times F_2$

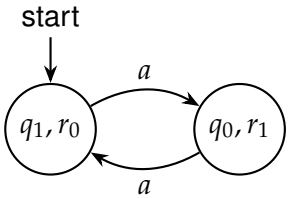


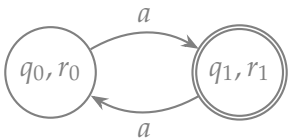


Both accept a^ω, so $\mathcal{L}(\mathcal{A}_1) \cap \mathcal{L}(\mathcal{A}_2) = \{a^\omega\}$.

Intersection of NBAs — Naive Approach

$Q_1 \times Q_2, \quad F = F_1 \times F_2 = \{(q_1, r_1)\}$:





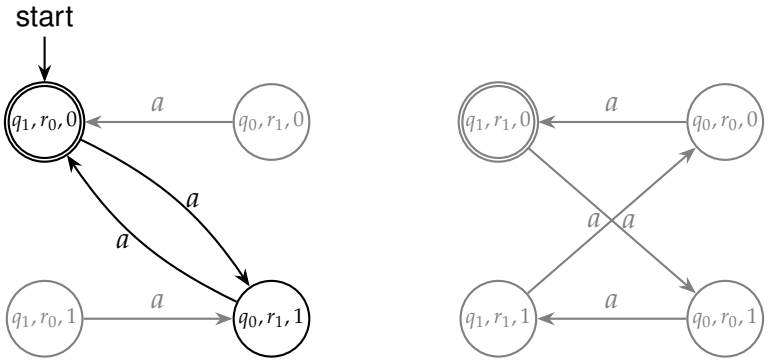
Intersection of NBAs

Two copies with index $c \in \{0, 1\}$.

- States: $Q = Q_1 \times Q_2 \times \{0, 1\}$
- Initial states: $I = I_1 \times I_2 \times \{0\}$
- Transitions: $(q_1, q_2, c) \xrightarrow{a} (q_1', q_2', c')$ iff
  - $q_1 \xrightarrow{a} q_1'$ in $\mathcal{A}_1$ and $q_2 \xrightarrow{a} q_2'$ in $\mathcal{A}_2$ (sync on letter $a \in \Sigma$)
  - $c' = next(q_1, q_2, c) = \begin{cases} 1 & c = 0 \text{ and } q_1 \in F_1 \\ 0 & c = 1 \text{ and } q_2 \in F_2 \\ c & \text{otherwise} \end{cases}$
- Accepting: $F = F_1 \times Q_2 \times \{0\}$

Copy 0 waits for F_1, then switches to copy 1 which waits for F_2, then back.
Infinitely many switches $\Rightarrow$ both F_1 and F_2 visited infinitely often.

Intersection of NBAs



Kripke Structure $\to$ Büchi Automaton

Transition system as a Kripke structure $M = (S, S_0, \to, L, \underline{\text{Atoms}})$:

- $S_0 \subseteq S$ — initial states
- $\underline{\text{Atoms}}$ — atomic propositions

Construct NBA $\mathcal{A}_M = (\mathcal{P}(\underline{\text{Atoms}}),\ S,\ S_0,\ \Delta,\ F)$:

- States $= S$, initial states $= S_0$ (reuse M's states)
- $F = S$ (all states accepting)
- Transition: $q \xrightarrow{a} p$ iff $q \to p$ and $L(q) = a$

Label on edge $q \to p$ is *source*'s label $L(q)$. Run $q_0\, q_1\, q_2\, \cdots$ on word $a_0 a_1 a_2 \cdots$ is valid iff $q_i \to q_{i+1}$ in M and $a_i = L(q_i)$.

$F = S \Rightarrow$ every infinite run accepts. $\mathcal{L}(\mathcal{A}_M) = \mathit{Traces}(M)$.

The LTL Model Checking Algorithm

Given M and φ:

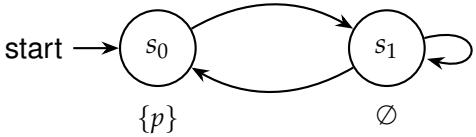
1. Translate $\neg\varphi$ into an NBA $\mathcal{A}_{\neg\varphi}$.
2. Convert M into an NBA $\mathcal{A}_M$.
3. Compute the product $\mathcal{A}_{\neg\varphi} \times \mathcal{A}_M$.
4. Check if $\mathcal{L}(\mathcal{A}_{\neg\varphi} \times \mathcal{A}_M) = \emptyset$.

Result:

- Empty $\Rightarrow M \models \varphi$.
- Non-empty $\Rightarrow$ an accepting run gives a counterexample path in M.

Example – Setup

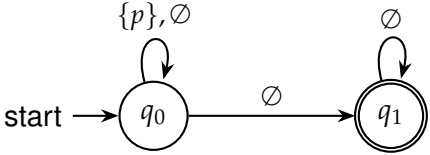
System M: <u>Atoms</u> $= \{p\}$



Property: $\varphi = \mathbf{G}\,\mathbf{F}\,p$ ("p holds infinitely often")

$\neg\varphi = \mathbf{F}\,\mathbf{G}\,\neg p$ ("eventually $\neg p$ holds forever")

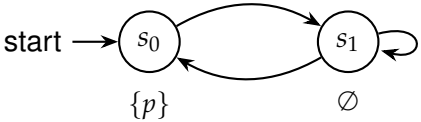
NBA for $\neg\varphi$:



Example – Converting M to NBA

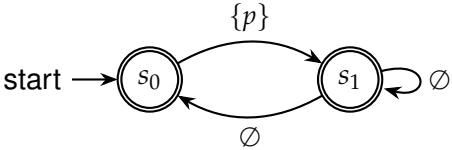
Put source state's label on each outgoing edge; make all states accepting:

System M:



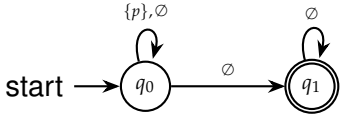
$\Rightarrow$

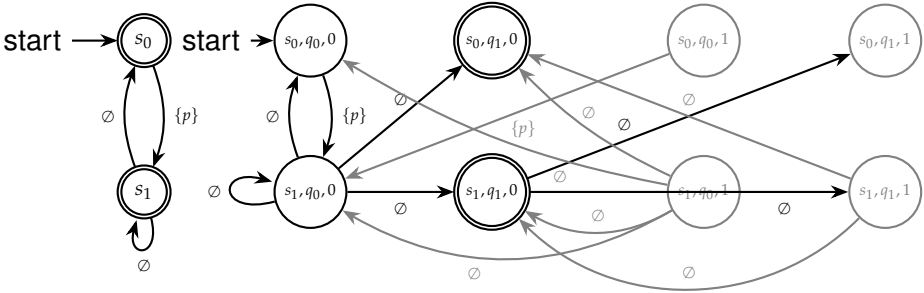
NBA $\mathcal{A}_M$:



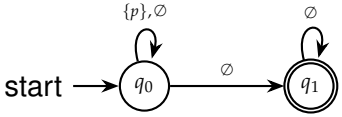
- Edge $q \to p$ gets label $L(q)$ (source's label).
- $\mathcal{A}_M$ accepts exactly the traces of M.

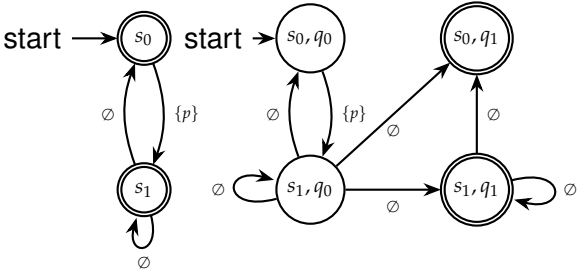
Example Product





Example Product





Counterexample: $s_0,\ s_1,\ s_1,\ s_1,\ \ldots\ \Rightarrow\ M \not\models \mathbf{G}\,\mathbf{F}\,p$.

Emptiness = Reachable Accepting Cycle

- $\mathcal{L}(\mathcal{A}) \neq \emptyset \iff \exists f \in F_{\mathcal{A}}$ that is reachable from an initial state and lies on a cycle.
- Checking $M \models \varphi$ is now a graph reachability problem on the product graph: find a reachable cycle through an accepting state.
- On-the-fly: product states generated lazily; early termination on counterexample.

TLA$^+$ toolbox

- Temporal Logic and Actions: mathematical foundation (formalism) for TLA$^+$
  - focused on actions (transitions of states)
  - temporal logic + set theory
  - developed by Leslie Lamport (author of LaTeX)

    > *I was sure that temporal logic was some kind of abstract nonsense that would never have any practical application, but it seemed like fun, so I attended [LTL seminar]. — Leslie Lamport*

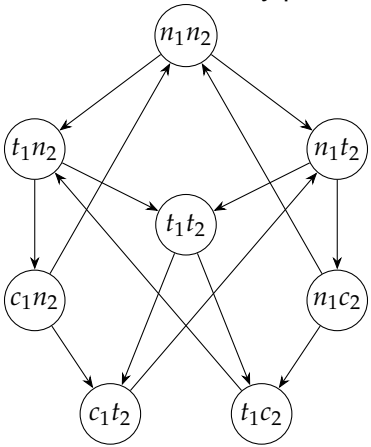
- TLA$^+$: specification language
  - similar to mathematical language
  - usually mostly ordinary non-temporal mathematics, which less cumbersome than a purely temporal specification
- TLC: model checker for TLA$^+$
  - finite model checking
  - generates a set of initial states
  - BFS over all defined state transitions (symmetry, parallelization, distribution)
- TLAPS: proof system to prove system properties
  - abstract proofs that are sent to back-end provers (Isabelle or SMT solvers like Z3; each with own limitation)

Rough mapping between LTL and TLA$^+$

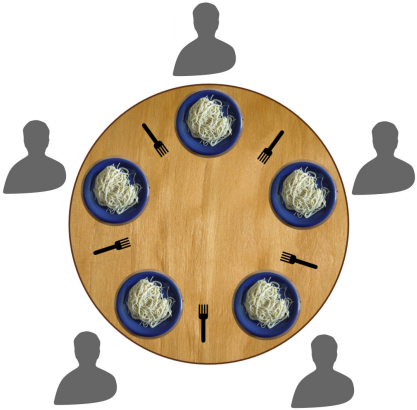
LTL	Meaning	TLA$^+$	Notes
$\neg\varphi$	not	~φ	
$\varphi \wedge \psi$	and	φ/\ψ	
$\varphi \vee \psi$	or	φ\/ψ	
$\varphi \Rightarrow \psi$	implication	φ => ψ	
$\varphi \Leftrightarrow \psi$	equivalence	φ == ψ	
$X\,\varphi$	next φ	φ'	Priming denotes next-state value (not to be used as temporal formula; this is how we define state transitions — actions)
$F\,\varphi$	eventually φ	<>	
$G\,\varphi$	always φ	[]φ	Prefer invariants if φ does not contain primes or temporal operators
$\varphi\,U\,\psi$	until		Not really needed
$\varphi \rightarrow F\psi$	leads to	φ~>ψ	

Mutex in TLA$^+$

Let's firstly look at Mutex we already presented in lectures:

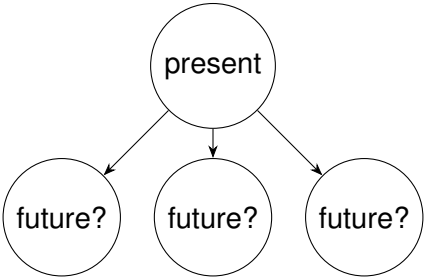


Dining philosophers problem



Each philosopher has a bowl of spaghetti and can reach the two forks on either side of them. Moreover, a philosopher can only eat their spaghetti when they have both a left and a right fork, so their neighbor cannot be eating at the same time. (Image by wikipedia:bdesham)

Computation tree logic (CTL)



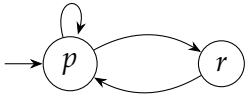
- Paradigm shift: branching future!
- Statements about states and paths/runs starting in states.
- **A**: “for all paths from the initial state”
- **E**: “for some path from the initial state”
- **X**: “for immediately next states”
- We already know what **G** and **F** mean for paths.

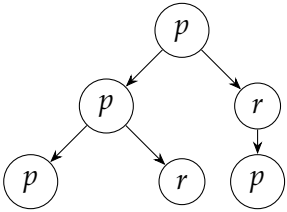
Syntax of CTL

$$
\begin{aligned}
\varphi, \psi ::= \; & p \\
\mid \; & \neg\varphi \\
\mid \; & \varphi \wedge \psi \\
\mid \; & \varphi \vee \psi \\
\mid \; & \mathbf{AX}\varphi && \varphi \text{ holds in all immediately next states} \\
\mid \; & \mathbf{EX}\varphi && \varphi \text{ holds in some immediately next state} \\
\mid \; & \mathbf{AG}\varphi && \varphi \text{ holds in all subsequent paths} \\
\mid \; & \mathbf{EG}\varphi && \varphi \text{ holds in some subsequent path} \\
\mid \; & \mathbf{AF}\varphi && \varphi \text{ holds in some state in all subsequent paths} \\
\mid \; & \mathbf{EF}\varphi && \varphi \text{ holds in some state in some subsequent path} \\
\mid \; & \mathbf{A}[\varphi \, \mathbf{U} \, \psi] && \varphi \text{ holds in all subsequent paths until } \psi \text{ holds} \\
\mid \; & \mathbf{E}[\varphi \, \mathbf{U} \, \psi] && \varphi \text{ holds in some subsequent path until } \psi \text{ holds}
\end{aligned}
$$

Models for CTL

- $\mathcal{M} = (S, \rightarrow, s, L)$ (a transition system + an initial state).
  - S is a set of states.
  - $\rightarrow$ is a *serial* transition relation on S (next step is always possible).
  - s is an initial state.
  - $L : S \rightarrow \mathcal{P}(\underline{\text{Atoms}})$ is a labelling function. $\underline{\text{Atoms}}$ is the set of propositional variables ($p \in \underline{\text{Atoms}}$).
- $\mathcal{M}$ uniquely determines an infinite tree with root s.
  - Idea: infinite branching-time progression of states starting in s.





Semantics for CTL

- $\mathcal{M}[s']$ is $\mathcal{M}$ with its initial state changed to s'.
- $\mathsf{pth}(s)$ is an infinite path in the transition graph starting in s.

$$
\begin{aligned}
\mathcal{M} \models p &\iff p \in L(s) \\
\mathcal{M} \models \mathbf{AX}\varphi &\iff \forall (s \to s').\mathcal{M}[s'] \models \varphi \\
\mathcal{M} \models \mathbf{EX}\varphi &\iff \exists (s \to s').\mathcal{M}[s'] \models \varphi \\
\mathcal{M} \models \mathbf{AG}\varphi &\iff \forall \mathsf{pth}(s) \forall s' \in \mathsf{pth}(s).\mathcal{M}[s'] \models \varphi \\
\mathcal{M} \models \mathbf{EG}\varphi &\iff \exists \mathsf{pth}(s) \forall s' \in \mathsf{pth}(s).\mathcal{M}[s'] \models \varphi \\
\mathcal{M} \models \mathbf{AF}\varphi &\iff \forall \mathsf{pth}(s) \exists s' \in \mathsf{pth}(s).\mathcal{M}[s'] \models \varphi \\
\mathcal{M} \models \mathbf{EF}\varphi &\iff \exists \mathsf{pth}(s) \exists s' \in \mathsf{pth}(s).\mathcal{M}[s'] \models \varphi \\
\mathcal{M} \models \mathbf{A}[\varphi \,\mathbf{U}\, \psi] &\iff \\
\forall \mathsf{pth}(s) \exists s' \in \mathsf{pth}(s).(\mathcal{M}[s'] \models \psi \text{ and } \forall (s'' < s').\mathcal{M}[s''] \models \varphi) & \\
\mathcal{M} \models \mathbf{E}[\varphi \,\mathbf{U}\, \psi] &\iff \\
\exists \mathsf{pth}(s) \exists s' \in \mathsf{pth}(s).(\mathcal{M}[s'] \models \psi \text{ and } \forall (s'' < s').\mathcal{M}[s''] \models \varphi) &
\end{aligned}
$$

Uses of CTL

- Note that CTL can only talk about branching paths — it is unable to talk about general properties of a path.
- Example: in (*extended*) LTL, $\mathbf{A}(\mathbf{G}(\mathbf{F}\varphi))$ means that in all future paths, φ holds infinitely often. The property "φ holds infinitely often in a path" cannot be expressed in CTL.
- Restricted syntax $\Rightarrow$ efficient model checking.
- Model checking for φ is linear in the size of φ.
- Consequence: CTL dominates as a specification language in industry.

Uses of CTL

Pattern	Category	CTL formula	Frequency
response	liveness	$\mathbf{AG}(\varphi \Rightarrow \mathbf{AF}\psi)$	43.4%
universality	safety	$\mathbf{AG}\varphi$	19.8%
absence	negated reachability	$\neg\mathbf{EF}\varphi$	7.4%
precedence	liveness	$\mathbf{AG}(\mathbf{A}[\neg\varphi\,\mathbf{W}\,\psi])$	4.5%
absence		$\mathbf{AG}((\varphi \wedge \neg\psi \wedge \mathbf{AF}\psi)$ $\Rightarrow \mathbf{A}[\neg\varphi'\,\mathbf{U}\,\psi])$	3.2%
absence	safety	$\mathbf{AG}(\varphi \Rightarrow \mathbf{AG}\neg\psi)$	2.1%
existence	liveness	$\mathbf{EF}\varphi$	2.1%

Table 1: Commonly used specification patterns for CTL.

- IBM RuleBase, Cadence SMV (hardware verification).

Real-World Applications of Model Checking

- Temporal logic model checking is used wherever correctness depends on *ordering* or *timing* of events across all possible executions.
- Key application domains:
  - **Hardware design** — cache coherence, bus protocols
  - **Concurrent software** — operating systems, autonomous agents
  - **Safety-critical systems** — avionics, medical devices, nuclear control
  - **Communication protocols** — network and security protocols
  - **Distributed systems** — cloud infrastructure, databases
- We look more closely at three case studies:
  - Spacecraft autonomy
  - Hardware protocols
  - Medical devices

Spacecraft Autonomy — NASA Deep Space 1

- **Domain:** concurrent autonomous spacecraft software
- **Case:** NASA Deep Space 1 Remote Agent
- **Why LTL:** path-by-path reasoning — if *this* failure occurs on *this* run, notification must eventually follow
- **Impact:** 5 concurrency bugs found pre-flight

[Hav+00]

Hardware Verification — IEEE Futurebus+

- **Domain:** concurrent hardware cache coherence protocols
- **Case:** IEEE Futurebus+ standard, verified with SMV
- **Why CTL:** branching-time guarantee — correctness must hold from *every* state across *all* futures, not just along one path
- **Impact:** errors found in the IEEE standard itself; up to 10^{30} states verified

[Cla+95]

Medical Devices — Pacemaker (UPPAAL)

- **Domain:** real-time cyber-physical systems
- **Case:** implantable dual-chamber pacemaker
- **Why TCTL:** real-valued clocks are needed; ordering alone is insufficient
- **Impact:** prevents unsafe pacing rhythms; detects pacemaker-mediated tachycardia

[Jia+12]

The Laureates

Edmund M. Clarke — Carnegie Mellon University

Developed symbolic model checking with BDDs; co-invented CTL; pioneered SAT-based bounded model checking.

E. Allen Emerson — University of Texas, Austin

Co-developed CTL and CTL*; proved decidability of CTL model checking; foundational automata-theoretic contributions.

Joseph Sifakis — VERIMAG / CNRS, France

Independent invention of model checking in France; contributed to real-time and hybrid system verification.

Awarded jointly in 2007 for developing model checking into a practical verification technique.

Timeline

- **1970s** — Hardware circuits growing rapidly. Correctness proofs done by hand: error-prone and unscalable.
- **1981** — Clarke & Emerson and Sifakis independently formulate model checking.
- **1982** — First published CTL model checking algorithm: $O(|S| \times |\varphi|)$ — polynomial in state space size.
- **1980s** — State explosion problem: real systems have a lot of states. Model checking seems impractical.
- **1990s** — BDDs & symbolic methods: allows checking 10^{20} states.
- **2000s** — SAT-based bounded model checking and CEGAR make industrial-scale verification routine.

The State Explosion Problem

The Core Difficulty

n boolean variables produce 2^n states. A circuit with 100 flip-flops has $2^{100} \approx 10^{30}$ states.

Key breakthroughs:

- Symbolic Model Checking (BDDs)
- Partial Order Reduction
- CEGAR (Counterexample-Guided Abstraction Refinement)
- Bounded Model Checking (SAT)

Key Breakthroughs: Part I

- **Symbolic Model Checking (BDDs)**
  - Introduced by Clarke and McMillan in 1987.
  - Uses **Binary Decision Diagrams** to represent sets of states.
  - Moves from checking individual states to checking logic formulas; handled 10^{20} states for the first time.

- **Partial Order Reduction**
  - Sometimes actions commute in synchronous systems.
  - Use this knowledge to reduce the number of interleavings we need to check.

Key Breakthroughs: Part II

- **CEGAR**
  - **C**ounter**e**xample-**G**uided **A**bstraction **R**efinement.
  - Core loop:
    1. Abstraction: model checker creates a simplified version of the system
    2. Verification: model checking on simplified version
    3. Validation: if found a counterexample, check it against the complex system
    4. Refinement: if counterexample isn't correct, refine the simplified model

- **Bounded Model Checking**
  - Uses **SAT Solvers**.
  - Instead of full proof, it searches for a counterexample within a fixed path length k.

4. The Industry Turning Point: Pentium FDIV

- **The Bug (1994):** A flaw in the lookup table of the Pentium floating-point divider.
- **The Cost:** $475 million recall.
- **The Result:** Software testing was no longer enough.

Conclusion

This disaster proved that "Formal Methods" were a financial necessity, not just an academic exercise.

Bibliography I

[Cla+18] Edmund M. Clarke et al., eds. *Handbook of Model Checking*. Cham: Springer, 2018.

[Cla+95] Edmund M. Clarke et al. “Verification of the Futurebus+ Cache Coherence Protocol”. In: *Formal Methods in System Design* 6 (1995), pp. 217–232. DOI: 10.1007/BF01383968.

[DAC99] Matthew B. Dwyer, George S. Avrunin, and James C. Corbett. “Patterns in Property Specification for Finite-State Verification”. In: *Proceedings of the 21st International Conference on Software Engineering (ICSE)*. IEEE Computer Society Press, 1999, pp. 411–420.

[Hav+00] Klaus Havelund et al. “Formal Analysis of the Remote Agent Before and After Flight”. In: *Proceedings of the 5th NASA Langley Formal Methods Workshop*. 2000.

Bibliography II

[Jia+12] Zhihao Jiang et al. “Closed-Loop Verification of Medical Devices with Model Abstraction and Refinement”. In: *International Journal on Software Tools for Technology Transfer* 14.2 (2012), pp. 191–213.

[Unk] Unknown. *Computation Tree Logic (CTL)*. http://cs112.org/wp-content/uploads/2016/09/CTL.pdf. Course notes, accessed April 2026.

[Var01] Moshe Y. Vardi. “Branching vs. Linear Time: Final Showdown”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Tiziana Margaria and Wang Yi. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–22. ISBN: 978-3-540-45319-2.